



① 简要介绍

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语



① 简要介绍

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语

词法定义 (Lexical Definitions)

词法分析器应将源代码识别为以下 Token 流:

① 关键字与字面量

- Keywords: if, else, while, read, put, write, get, func, global, local, return, break, continue
- Integer: $[0-9]^+$
- Identifier: $[a-zA-Z_][a-zA-Z0-9_]^*$
- CharConstant: ' (任何字符或转义字符 `\n`, `\\`, `\'`) '
- StringConstant: " (任何字符或转义字符序列) "

② 运算符与分隔符

- Operators: `+`, `-`, `*`, `/`, `%`, `<`, `<=`, `>`, `>=`, `==`, `!=`, `=`
- Delimiters: `(`, `)`, `[`, `]`, `,`, `;`

③ 辅助项

- Comment: `#` 之后至行尾的所有内容 (忽略)
- Whitespace: 空格、制表符、换行符 (作为 Token 分隔符, 本身无语义)



① 简介

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语

语法定义 (Grammar Definitions)

Listing 1: 顶层结构

```

1 Program      ::= { FunctionDef | Statement }
2
3 FunctionDef   ::= "func" [ "[" ] Identifier "(" [ ParameterList ] ")" Block
4
5 ParameterList ::= Identifier { "," Identifier }
```

语句 (续)

Listing 2: 语句

```

1 Statement      ::= AssignmentStmt
2                | IfStmt
3                | WhileStmt
4                | JumpStmt
5                | DeclarationStmt
6                | Block
7                | ";"
8
9 Block          ::= "{" { Statement } "}"
10
11 IfStmt         ::= "if" "(" Expression ")" Block "else" Block
12
13 WhileStmt      ::= "while" "(" Expression ")" Block
14
15 AssignmentStmt ::= LValue "=" Expression ";"
16                | Identifier "[" Expression "]" "=" Expression ";" /* 数组声明或初始化 */

```

语句

Listing 3: 语句 (续)

```

1  JumpStmt      ::= "break" ";"
2                  | "continue" ";"
3                  | "return" Expression ";"
4
5  DeclarationStmt ::= ( "local" | "global" ) Identifier [ "[" "]" ] ";"

```

表达式

Listing 4: 表达式

```

1  LValue          ::= Identifier [ "[" Expression "]" ]
2                    | "write"
3                    | "put"
4
5  Expression      ::= LogicalExp
6
7  LogicalExp      ::= RelationalExp { ("==" | "!=") RelationalExp }
8
9  RelationalExp   ::= AdditiveExp { ("<" | "<=" | ">" | ">=") AdditiveExp }
10
11 AdditiveExp      ::= MultiplicativeExp { ("+" | "-") MultiplicativeExp }
12
13 MultiplicativeExp ::= Primary { ("*" | "/" | "%") Primary }

```


表达式 (续)

Listing 5: 表达式 (续)

```

1 Primary      ::= Integer
2               | CharConstant
3               | StringConstant
4               | Identifier "(" [ ArgumentList ] ")" /* 函数调用 */
5               | LValue
6               | "read"
7               | "get"
8               | "(" Expression ")"
9
10 ArgumentList ::= Expression { "," Expression }
```



① 简要介绍

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语

语义约束与补充说明——变量推断

全局变量或局部变量在**首次**声明或使用时根据上下文推断类型，具体规则如下：

- 有以下情况之一的，推断为整数变量：
 - 作为右值的，如 $a = b + 1$ ；中的 b ，推断为全局整数变量并赋予初始值 0。
 - $a = \text{expr};$ 、 $\text{local } a;$ 或 $\text{global } a;$ 且 expr 是结果为整数的表达式（字面量、变量、函数调用等）。
 - $a = \text{read};$ 且 a 未定义，则等价于先执行 $\text{global } a;$ 再执行 $a = \text{read};$ ，类型推断为整数变量。
- 有以下情况之一的，推断为数组变量：
 - $a = \text{expr};$ 、 $\text{local } a[];$ 或 $\text{global } a[];$ 且 expr 是结果为数组的表达式（函数调用、字符串字面值）。
 - $a[] = \text{expr};$ 且 expr 是结果为整数的表达式（字面值、变量、函数调用等）。
 - 如 $a[] = \text{read};$ 且 a 未定义，则等价于先执行 $\text{global } a[];$ 再执行 $a[] = \text{read};$ ，类型推断为数组变量¹。

¹特别地， $a = \text{"Hello"};$ 与 $a[] = \text{"Hello"};$ 均推断为数组变量 a ，并根据字符串长度动态分配内存存储 UTF-32 编码的 C 风格字符串内容。

语义约束与补充说明（续）

- ③ 内存寻址逻辑：由于统一采用 UTF-32 编码和 32 位整数，所有数组索引 $a[i]$ 的底层地址计算逻辑为： $\text{Address} = \text{Base}(a) + (i \times 4)$ 字符串常量在数据段或堆区分配时，需确保每个字符占用 4 字节，并以 0x00000000 结尾。
- ④ 符号表设计建议
- Name: 标识符名称。
 - Type: INT 或 ARRAY_PTR。
 - Scope: GLOBAL 或 LOCAL（及嵌套层级索引）。
 - Offset/Address: 在栈帧中的偏移或全局区的地址。

Mandrill 示例代码

```
1 write = read + 1;
2 put = '\n';
```

输入

5

输出

6

Mandrill 示例代码 2

```

1  a = read;
2  b = read;
3  if (b > a) {
4      c = a;
5      a = b;
6      b = c;
7  }
8  while (a % b != 0) {
9      c = b;
10     b = a % b;
11     a = c;
12 }
13 write = b;
14 put = '\n';
    
```

输入

60 36

输出

12

Mandrill 示例代码 3

```
1 func strlen(s[]) {
2     local i; i = 0;
3     while (s[i] != 0) { i = i + 1; }
4     return i; }
5 func[] strcpy(a[], b[]) {
6     local la; local lb;
7     la = strlen(a); lb = strlen(b);
8     local c[]; c[] = la + lb + 1;
9     local i; i = 0;
10    while (i < la) {
11        c[i] = a[i]; i = i + 1; }
12    while (i < la + lb) {
13        c[i] = b[i - la]; i = i + 1; }
14    c[la + lb + 1] = 0; return c; }
15 a[] = "Hello_"; b[] = "World\n";
16 c[] = strcpy(a, b);
17 write = c;
```

输出

Hello World